

Programación Avanzada

Interfaces de usuarios

Índice

Esquema	3
Ideas clave	4
5.1. Introducción y objetivos	4
5.2. AWT	4
5.3. Gestores de posicionamiento	6
5.4. ¿Qué son los eventos?	12
5.5. Adaptadores	13
A fondo	14
Test	19



5.1. Introducción y objetivos

Para estudiar este tema debes leer las páginas 162-176 del siguiente libro, disponible en la Biblioteca Virtual de UNIR:

Sznajdleder, P. A. (2013). «Interfaz gráfica (GUI)». En Autor, *Java a Fondo: estudio del lenguaje y desarrollo de aplicaciones* (2ª ed.) (pp.162-176). Buenos Aires: Alfaomega Grupo Editor.

En este tema vamos a ver en profundidad cómo se gestionan los elementos visuales dentro de Java. Veremos:

- ▶ Introducción a AWT.
- ▶ Organización de los elementos visuales.

5.2. AWT

Una interfaz gráfica (GUI, *Graphical User Interface*) es un conjunto de componentes gráficos que posibilitan la interacción entre el usuario y la aplicación. Estos componentes se encuentran en el paquete `java.awt` (*Abstract Window Toolkit*).

En la jerarquía de clases de `awt`, tienen especial importancia las clases `Component` y `Container`.

Toda clase que hereda de la clase `Component` es un componente y toda clase que hereda de la clase `Container` es un contenedor (área en la que se pueden colocar componentes).

Jerarquía de clases `Component`:

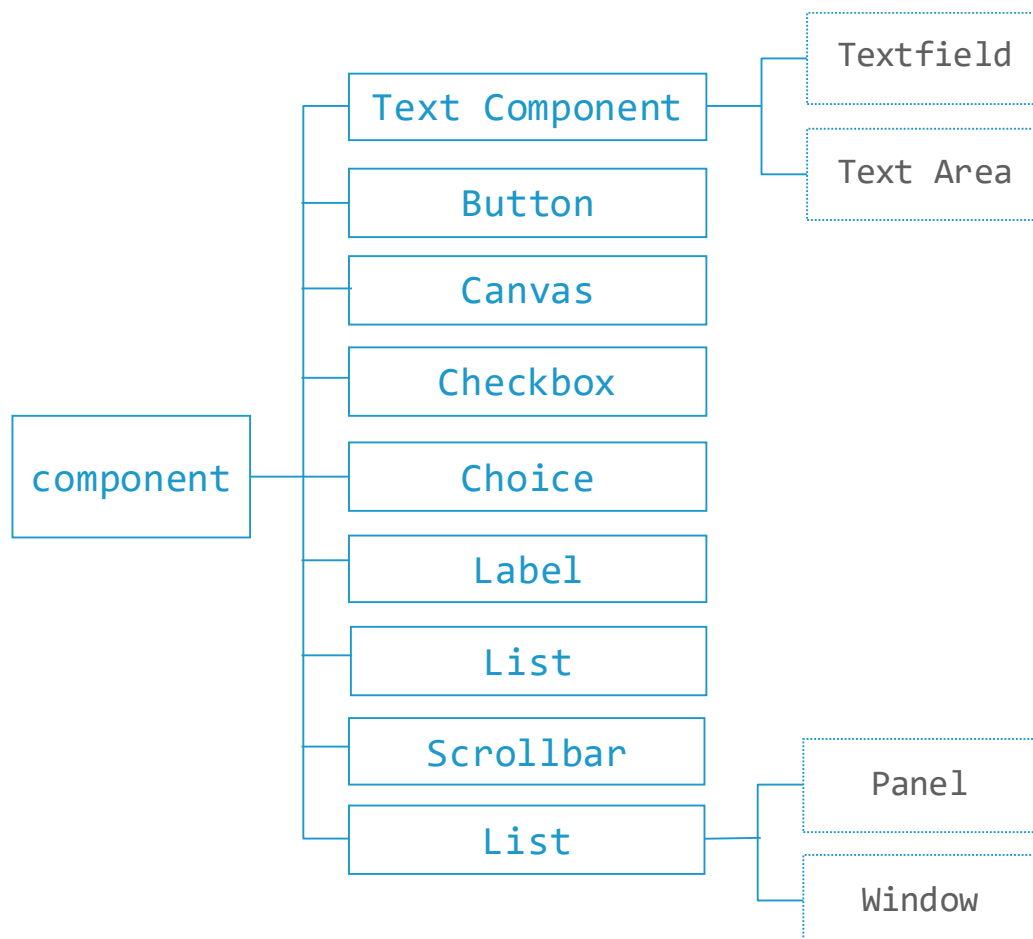


Figura 1. Jerarquía de clases `Component`.

Para añadir más componentes y situarlos en una posición exacta, se utiliza un **gestor de posicionamiento o administrador** de diseños, mediante la interfaz `LayoutManager` del paquete `java.awt`.

Las siguientes clases implementan la interfaz `LayoutManager`

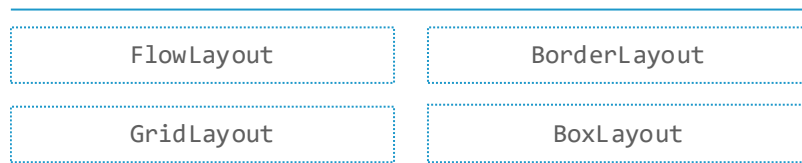


Figura 2. Interfaz `LayoutManager`.

El método `setLayout(LayoutManager m)` permite establecer el gestor de posicionamiento. El argumento de este método es un objeto de alguna de las clases que implementan la interfaz `LayoutManager`.

5.3. Gestores de posicionamiento

FlowLayout

Es el **administrador de diseños más básico**, en el que los componentes se colocan de izquierda a derecha, dejando, por omisión, un espacio vertical y horizontal de 5 píxeles entre sus componentes.

Constructores:

- ▶ `FlowLayout()`: construye el objeto `FlowLayout` con alineación central.
- ▶ `FlowLayout(int alineación)`: construye el objeto `FlowLayout` con la alineación que se le pasa como argumento (`RIGHT`, `LEFT`, `CENTER`).
- ▶ `FlowLayout(int alineación, int distancia_h, int distancia_v)`: construye el objeto `FlowLayout` con la alineación indicada en el primer argumento. La distancia entre los componentes se indica en píxeles con los dos últimos argumentos.

Ejemplo tomado del manual de referencia indicado al principio del tema:

```

package libro.cap05;
import java.awt.*;

public class Ventanal extends Frame
{
    // defino tres objetos Button
    private Button b1,b2,b3;

    public Ventanal()
    {
        // el constructor del padre recibe el titulo de la ventana
        super("Esta es la Ventana 1");

        // defino el layout que va a tener la ventana: FlowLayout
        setLayout(new FlowLayout());

        // instancio el primer boton y lo agrego al container
        b1=new Button("Boton 1");
        add(b1);

        // instancio el segundo boton y lo agrego al container
        b2=new Button("Boton 2");
        add(b2);

        // instancio el tercer boton y lo agrego al container
        b3=new Button("Boton 3");
        add(b3);

        // defino el size de la ventana y la hago visible
        setSize(300,300);
        setVisible(true);
    }

    public static void main(String[] args)
    {
        Ventanal v1 = new Ventanal();
    }
}

```



Figura 3. Ejemplo de FlowLayout (Sznajdleder, 2013)

BorderLayout

Este gestor de posicionamiento coloca los componentes en cinco zonas (NORTH, SOUTH, EAST, WEST y CENTER). Por omisión se colocan en el centro.

Constructores:

- `BorderLayout()`: construye el objeto `BorderLayout` sin separación entre los componentes.

- ▶ `BorderLayout(int distancia_h, int distancia_v)`: construye un objeto `BorderLayout` con una separación entre los componentes indicada en píxeles en los dos argumentos del constructor.

Método `add`:

El método `add` permite añadir componentes. Su sintaxis es:

```
add(Component comp, int area)
```

El argumento área indica donde se sitúa el componente. Los valores posibles son: `BorderLayout.CENTER`, `BorderLayout.NORTH`, `BorderLayout.SOUTH`, `BorderLayout.WEST` y `BorderLayout.EAST`.

Ejemplo tomado del manual de referencia:

```
package libro.cap05;
import java.awt.*;
public class Ventana2 extends Frame
{
    private Button bNorth,bSouth,bWest,bEast,bCenter;

    public Ventana2()
    {
        super("Esta es la Ventana 2");
        setLayout(new BorderLayout());

        bNorth=new Button("Norte");
        add(bNorth, BorderLayout.NORTH);

        bSouth=new Button("Sur");
        add(bSouth, BorderLayout.SOUTH);

        bWest=new Button("Oeste");
        add(bWest, BorderLayout.WEST);

        bEast=new Button("Este");
        add(bEast, BorderLayout.EAST);

        bCenter=new Button("Centro");
        add(bCenter, BorderLayout.CENTER);

        setSize(300,300);
        setVisible(true);
    }

    public static void main(String[] args)
    {
        Ventana2 v2 = new Ventana2();
    }
}
```

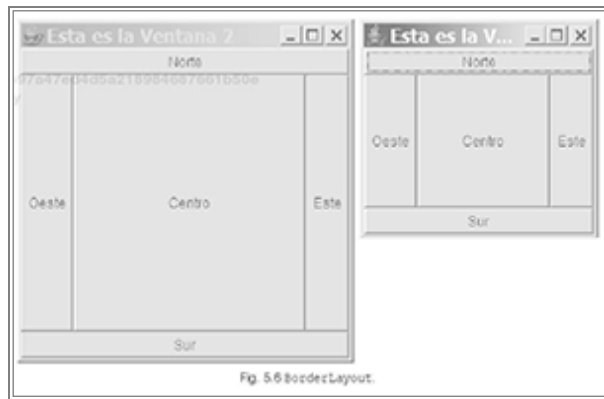



Figura 4. Ejemplo de BorderLayout (Sznajdleder, 2013).

GridLayout

El gestor de posicionamiento GridLayout coloca los componentes en filas y columnas.

Todos los componentes tendrán el mismo tamaño y se comienzan a colocar en la celda superior izquierda, avanzando de izquierda a derecha hasta que se completa la fila. Luego se continúa por la siguiente fila.

Constructores:

- ▶ `GridLayout()`: construye el objeto GridLayout con una única fila y una única columna.
- ▶ `GridLayout(int filas, int columnas)`: construye el objeto GridLayout el número de filas y columnas indicados en los argumentos.
- ▶ `GridLayout(int filas, int columnas, int distancia_h, int distancia_v)`: Construye el objeto GridLayout con el número de filas y columnas indicados en los dos primeros argumentos y con una separación horizontal y vertical entre los componentes indicada en pixeles en los dos últimos argumentos.

Ejemplo tomado del manual de referencia:



```
package libro.cap05;
import java.awt.*;
public class Ventana3 extends Frame
{
    private Button b1,b2,b3,b4,b5,b6,b7,b8,b9;
    public Ventana3()
    {
        super("Esta es la Ventana 3");
        setLayout(new GridLayout(3,3) );

        b1=new Button("Boton 1");
        add(b1);

        b2=new Button("Boton 2");
        add(b2);

        b3=new Button("Boton 3");
        add(b3);

        b4=new Button("Boton 4");
        add(b4);

        b5=new Button("Boton 5");
        add(b5);

        b6=new Button("Boton 6");
        add(b6);

        b7=new Button("Boton 7");
        add(b7);

        b8=new Button("Boton 8");
        add(b8);

        b9=new Button("Boton 9");
        add(b9);

        setSize(300,300);
        setVisible(true);
    }

    public static void main(String[] args)
    {
        Ventana3 v3 = new Ventana3();
    }
}
```

Figura 5. Ejemplo de GridLayout Fuente: Sznajdleder, P.A. (2013)

BoxLayout

El gestor de posicionamiento `BoxLayout` coloca los componentes en una única fila o columna dentro de un objeto de la clase `Container`.

Constructor:

`BoxLayout(Container destino, int orientación)`

Siendo `destino` el contenedor a utilizar y `orientación` indica si los componentes se colocan en fila o en columna. Los valores posibles de orientación son: `BoxLayout.X_AXIS` o `BoxLayout.Y_AXIS`.

Ejemplo:

```
Frame frame = new Frame();
frame.setTitle("BoxLayoutprueba");
Panel panel1 = new Panel();
panel1.setLayout(new BoxLayout(panel1, BoxLayout.Y_AXIS));
panel1.add(new Button("Button 1"));
panel1.add(new Button("Button 2"));
panel1.add(new Button("Button 3"));
panel1.add(new Button("Button 4"));
panel1.add(new Button("Button 5"));
panel1.add(new Button("Button 6"));
```

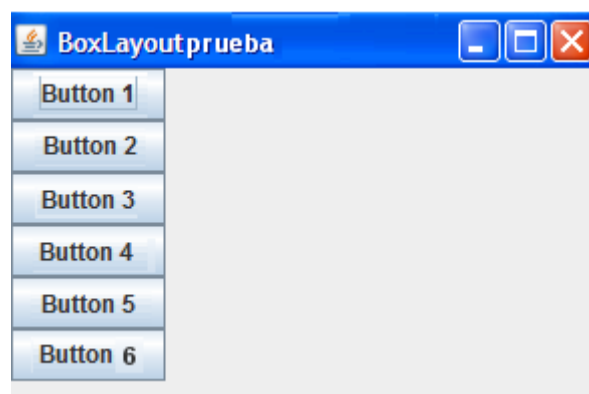


Figura 6. Ejemplo de `BoxLayout`.

La clase box

La clase Box define contenedores que tienen como gestor de posicionamiento un BorderLayout.

Combinar varios gestores

Una GUI se puede construir **combinando varios gestores de posicionamiento**. Cuando se utilizan varios gestores de posicionamiento se deben tener en cuenta los siguientes puntos:

- ▶ En contenedor solo se puede utilizar un tipo de gestor.
- ▶ Un contenedor de primer nivel puede tener otros contenedores anidados, para ello se utilizan objetos JPanel.

En el material de estudio asociado a este tema tienes algunos ejemplos en los que podrás encontrar el uso de distintos gestores de posicionamiento.

5.4. ¿Qué son los eventos?

Los **eventos** que se generan dentro de las interfaces gráficas deben ser gestionados para poder responder a ellos.

Los eventos **siempre tienen una fuente**, que es el objeto que es capaz de detectar el evento y notificar a los receptores que se ha producido. Por otro lado, debe existir un receptor (Listener) que será el encargado de gestionar el evento.

En el material de estudio tienes distintos ejemplos que te resultarán de utilidad para entender su funcionamiento.

5.5. Adaptadores

El **problema** cuando utilizamos Listeners es que **debemos implementar todos los métodos**, para evitarlo algunas interfaces tienen lo que se denominan **adaptadores**.

Adaptadores

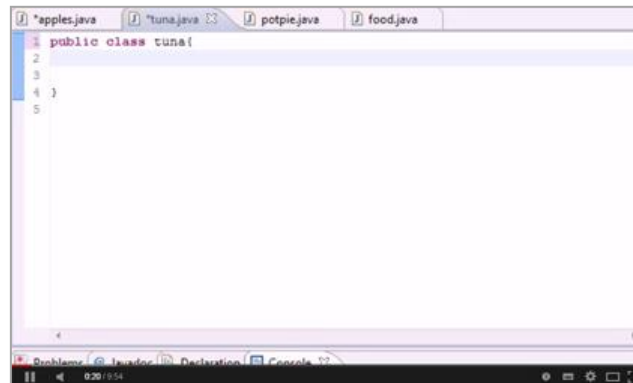
Son clases que proporcionan métodos vacíos para cada uno de los métodos de la interfaz.

A continuación, se detallan algunos adaptadores y la *interface* que implementan (adaptador – Listener):

- ▶ ComponentAdapter - ComponentListener
- ▶ ContainerAdapter - ContainerListener
- ▶ FocusAdapter - FocusListener
- ▶ KeyAdapter - KeyListener
- ▶ MouseAdapter - MouseListener
- ▶ MouseMotionAdapter - MouseMotionListener
- ▶ WindowAdapter - WindowListener
- ▶ WindowAdapter - WindowFocusListener

Crear interfaces de usuario

Vídeo en el que se explica cómo crear interfaces de usuario.

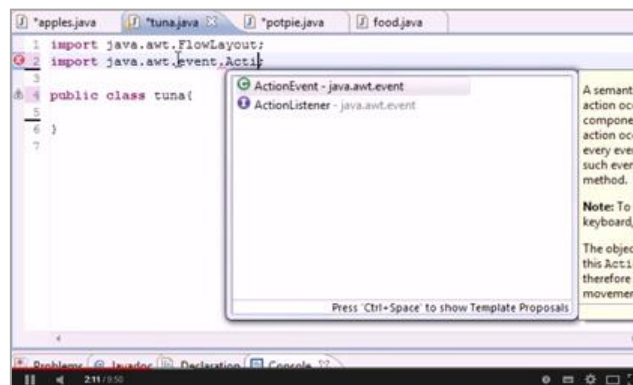


Accede al vídeo a través del aula virtual o desde la siguiente dirección web:

<http://www.youtube.com/watch?v=jUdIAgJ7JKo>

Gestionar eventos en Java

Vídeo en el que se explica cómo gestionar eventos en Java.

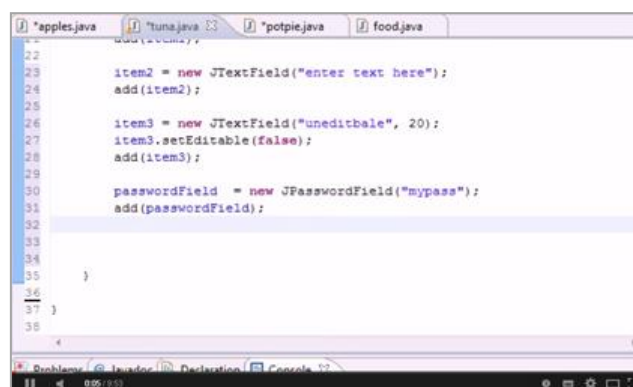


Accede al vídeo a través del aula virtual o desde la siguiente dirección web:

<http://www.youtube.com/watch?v=3EE7E3bvfe8>

Crear eventos en Java

Vídeo en el que se explica cómo crear eventos en Java.



Accede al vídeo a través del aula virtual o desde la siguiente dirección web:

<http://www.youtube.com/watch?v=ghYook53oIE>

Clase Box

En la página de Oracle podrás encontrar información detallada sobre la clase Box.

Accede al artículo a través del aula virtual o desde la siguiente dirección web:

<https://docs.oracle.com/javase/7/docs/api/javax/swing/Box.html>

Desarrollo y generación de interfaces de usuario

Lozano, M. D., González, P., Ramos, I., Moreno, F. y Molina, J. P. (2002). Desarrollo y generación de interfaces de usuario a partir de técnicas de análisis de tareas y casos de uso. *Inteligencia Artificial. Revista Iberoamericana de Inteligencia Artificial*, 6(16).

Paper en el que se explican diferentes técnicas para el desarrollo de interfaces de usuario.

Accede al artículo a través del aula virtual o desde la siguiente dirección web:

<http://www.redalyc.org/articulo.oa?id=92561610>

Verificar interfaces de usuario

Mehlitz, P., NASA Ames Res., Tkachuk, O. y Ujma, M. (2011). JPF-AWT: Model checking GUI applications. Automated Software Engineering (ASE).

Paper en el que se explica cómo verificar interfaces de usuario.

Accede al artículo a través del aula virtual o desde la siguiente dirección web:

https://ti.arc.nasa.gov/m/groups/rse/papers/Mehlitz_jpf-awt.pdf

Uso de componentes en SWING

Oracle (2019). « Lesson: Using Swing Components». *The Java Tutorials*.



Tutorial sobre la utilización de componentes en SWING.

Accede al tutorial a través del aula virtual o desde la siguiente dirección web:

<http://docs.oracle.com/javase/tutorial/uiswing/components/>

Gestores de posicionamiento

Oracle (2019). «Lesson: Laying Out Components Within a Container». *The Java Tutorials*.



Información detallada sobre los distintos gestores de posicionamiento.

Accede al tutorial a través del aula virtual o desde la siguiente dirección web:

<https://docs.oracle.com/javase/tutorial/uiswing/layout/>

Eclipse

Eclipse es una plataforma de desarrollo. Fue concebida desde sus orígenes para convertirse en una plataforma de integración de herramientas de desarrollo. No tiene en mente un lenguaje específico, sino que es un IDE genérico.



Accede a la página a través del aula virtual o desde la siguiente dirección web:

<https://www.eclipse.org/downloads/>

1. Para establecer el gestor de posicionamiento en un contenedor se debe:
 - A. Utilizar el método `setLayout`.
 - B. Indicar en el constructor del gestor.
 - C. Se indica al final de la construcción del objeto.
 - D. No existen los gestores de posicionamiento.

2. A través del gestor de contenido `FlowLayout`:
 - A. Los componentes fluyen de arriba abajo.
 - B. Los componentes fluyen de derecha a izquierda.
 - C. Los componentes fluyen de izquierda a derecha.
 - D. Los componentes fluyen de abajo a arriba.

3. El gestor de contenidos `BorderLayout`:
 - A. Divide el contenedor en cuatro zonas, norte, sur, este y oeste.
 - B. Divide el contenedor en tantas partes como se quieran.
 - C. Divide el contenedor en cinco partes, norte, sur, este, oeste y centro.
 - D. Divide el contenedor en dos partes.

4. El gestor de contenidos `GridLayout`:
 - A. Los componentes se colocan en filas.
 - B. Los componentes se colocan en orden de aparición.
 - C. Los componentes se colocan por coordenadas.
 - D. Los componentes se colocan en una rejilla de celdas iguales.

5. El gestor de contenidos `BoxLayout`:
- A. Muestra los componentes en una única fila o columna.
 - B. Muestra los componentes en tantas filas o columnas como queramos.
 - C. Muestra los componentes en una fila.
 - D. Muestra los componentes en una columna.
6. ¿Qué afirmación es correcta?
- A. Un contenedor puede tener tantos gestores como se quieran.
 - B. Un contenedor solo puede tener un gestor.
 - C. Un contenedor puede usar varios gestores, pero no a la vez.
 - D. Un contenedor no tiene gestores.
7. ¿Qué afirmación es correcta?
- A. Los eventos pueden no tener fuente.
 - B. Los eventos son generados por el `Listener`.
 - C. Los eventos se crean siempre a través de la instanciación del objeto `Action`.
 - D. Los eventos siempre tienen una fuente.
8. En Java:
- A. Se pueden crear `Listener` propios.
 - B. Se pueden crear `Listener` propios pero siempre que se definan en el constructor.
 - C. No se pueden crear `Listener` propios.
 - D. Todas las clases son `Listener`.
9. Los adaptadores se utilizan para:
- A. Gestionar objetos de tipo `action`.
 - B. Para simplificar la creación de `Listener` propios.
 - C. Para adaptar los programas a las nuevas versiones de eventos.
 - D. Para hacer compatibles los orígenes de los eventos y los `Listener`.

10. Los adaptadores en Java:

- A. Tienen el mismo nombre que el `Listener`.
- B. Tienen el mismo nombre que el `Listener` pero con `Adapter` al final en lugar de `Listener`.
- C. Tienen un nombre diferente al `Listener`.
- D. Los debe crear el usuario.